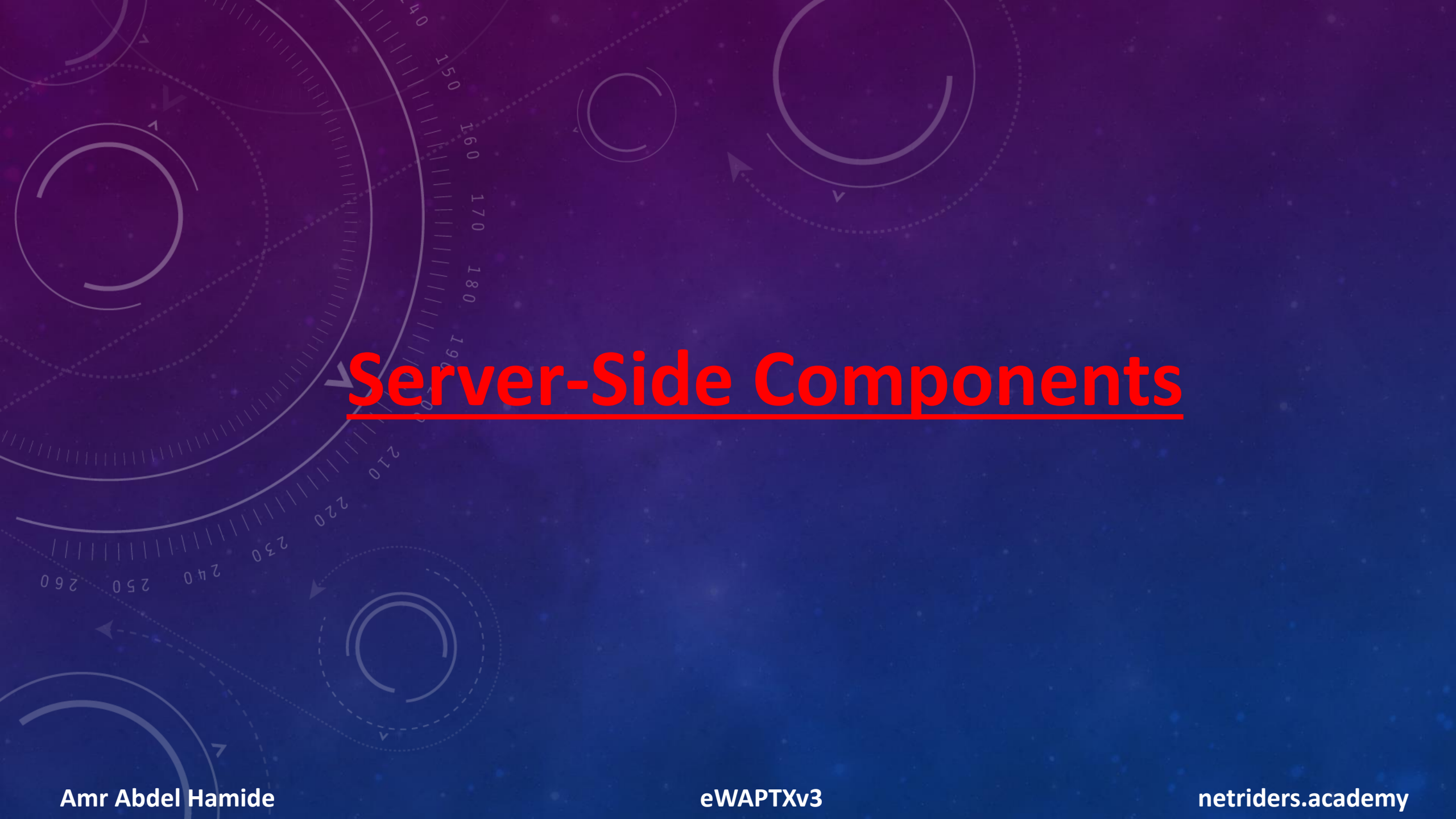


The background is a dark blue gradient with a subtle pattern of small white dots. Overlaid on this are several faint, light blue circular elements. These include concentric circles of varying sizes, some with dashed lines, and circular arcs with tick marks and degree labels (e.g., 140, 150, 160, 170, 180, 190, 200, 210, 220, 230, 240, 250, 260). Some of these circles have arrows indicating a clockwise direction.

Server-Side Components

1. Web Server

The web server is the cornerstone of the server-side architecture. Its primary responsibility is to receive HTTP requests from clients (typically web browsers) and serve the appropriate resources in response.

•Functionality:

- **Request Handling:** Accepts incoming HTTP requests (GET, POST, PUT, DELETE, etc.).
- **Static Content Serving:** Delivers static files like HTML, CSS, JavaScript, images, and videos directly from the file system.
- **Dynamic Content Processing:** Delegates requests for dynamic content to application servers or other backend components.
- **Security:** Implements security measures like SSL/TLS encryption, authentication, and authorization.
- **Load Balancing:** Distributes incoming traffic across multiple application servers to improve performance and availability.
- **Logging:** Records server activity for monitoring and debugging purposes.

•Examples:

- **Apache HTTP Server:** A widely used, open-source web server known for its flexibility and extensive module support.
- **Nginx:** Another popular open-source web server, often used as a reverse proxy, load balancer, and HTTP cache.
- **Microsoft IIS (Internet Information Services):** A web server developed by Microsoft, primarily used in Windows environments.

2. Application Server

The application server is responsible for executing the application logic and generating dynamic content. It acts as an intermediary between the web server and the database.

•Functionality:

- **Business Logic Execution:** Executes the core business logic of the application, written in languages like Java, Python, PHP, Ruby, or Node.js.
- **Data Processing:** Processes data received from the web server or other sources, performing calculations, transformations, and validations.
- **Database Interaction:** Interacts with the database to retrieve, store, and update data.
- **Session Management:** Manages user sessions, storing information about logged-in users and their activities.
- **Templating:** Generates dynamic HTML pages by combining data with templates.
- **API Endpoints:** Exposes API endpoints for other applications or services to interact with.

•Examples:

- **Java-based:** Tomcat, Jetty, JBoss/WildFly, GlassFish
- **Python-based:** Gunicorn, uWSGI
- **Node.js:** Express.js, Koa.js
- **PHP-based:** Apache with mod_php, PHP-FPM
- **Ruby-based:** Puma, Unicorn

3. Database Server

The database server is responsible for storing and managing the application's data. It provides a structured way to organize, access, and manipulate data.

•Functionality:

- **Data Storage:** Stores data in tables, documents, or other data structures.
- **Data Retrieval:** Allows applications to retrieve data using queries.
- **Data Manipulation:** Provides mechanisms for inserting, updating, and deleting data.
- **Data Integrity:** Enforces data integrity constraints to ensure data accuracy and consistency.
- **Concurrency Control:** Manages concurrent access to data to prevent conflicts.
- **Backup and Recovery:** Provides mechanisms for backing up and restoring data in case of failures.

•Types:

- **Relational Databases (RDBMS):** Store data in tables with rows and columns, using SQL for querying. Examples: MySQL, PostgreSQL, Oracle, Microsoft SQL Server.
- **NoSQL Databases:** Offer more flexible data models and are often used for handling large volumes of unstructured or semi-structured data. Examples: MongoDB, Cassandra, Redis, Couchbase.

4. Caching Server

The caching server stores frequently accessed data in memory to improve performance and reduce the load on the database.

•Functionality:

- **Data Caching:** Stores copies of data that are frequently accessed by the application.
- **Cache Invalidation:** Removes outdated data from the cache to ensure data consistency.
- **Cache Expiration:** Sets expiration times for cached data to prevent stale data from being served.
- **Distributed Caching:** Distributes the cache across multiple servers to improve scalability and availability.

•Examples:

- **Redis:** An in-memory data structure store, often used as a cache, message broker, and database.
- **Memcached:** A distributed memory object caching system, primarily used for speeding up dynamic web applications.

5. Message Queue

The message queue provides asynchronous communication between different components of the application. It allows components to send messages to each other without having to wait for a response.

•Functionality:

- **Message Queuing:** Stores messages in a queue until they are processed by a consumer.
- **Asynchronous Communication:** Enables components to communicate asynchronously, improving performance and scalability.
- **Decoupling:** Decouples components from each other, making the application more resilient to failures.
- **Reliable Delivery:** Ensures that messages are delivered reliably, even if components fail.

•Examples:

- **RabbitMQ:** A widely used open-source message broker.
- **Apache Kafka:** A distributed streaming platform, often used for building real-time data pipelines and streaming applications.
- **Amazon SQS (Simple Queue Service):** A fully managed message queuing service provided by AWS.

6. Load Balancer

The load balancer distributes incoming traffic across multiple servers to improve performance, availability, and fault tolerance.

•Functionality:

- **Traffic Distribution:** Distributes incoming traffic across multiple servers based on various algorithms (e.g., round robin, least connections).
- **Health Checks:** Monitors the health of servers and removes unhealthy servers from the pool.
- **Session Persistence:** Ensures that requests from the same user are routed to the same server.
- **SSL Termination:** Terminates SSL/TLS connections, reducing the load on the application servers.

•Examples:

- **Nginx:** Can be used as a load balancer in addition to its web server capabilities.
- **HAProxy:** A high-performance load balancer and reverse proxy.
- **Amazon ELB (Elastic Load Balancing):** A fully managed load balancing service provided by AWS.

7. API Gateway

The API gateway acts as a single entry point for all API requests. It provides a centralized point for managing and securing APIs.